Circuit Breaker - Complete Documentation

Table of Contents

1. Overview
2. Architecture
3. Installation & Setup
4. Quick Start Guide
5. API Reference
6. Configuration Guide
7. Monitoring & Observability
8. Testing Guide
9. Performance Benchmarks
10. Troubleshooting
11. Best Practices
12. Advanced Usage

Overview

What Problem Does This Solve?

In distributed systems, when a service becomes slow or unresponsive, continuing to send requests can:

- Exhaust connection pools
- Consume threads waiting for timeouts
- Cascade failures to upstream services
- Degrade overall system performance

Circuit Breaker solves this by:

1. Detecting when a service is failing
2. Stopping requests to that service (fail-fast)
3. Giving the service time to recover
4. Automatically retrying when appropriate

Real-World Example

```
Without Circuit Breaker:
User Request → API Gateway → Payment Service (down, 30s timeout)
                           → Payment Service (down, 30s timeout)
                           → Payment Service (down, 30s timeout)
Result: 90 seconds wasted, 3 threads blocked, user frustrated

With Circuit Breaker:
```

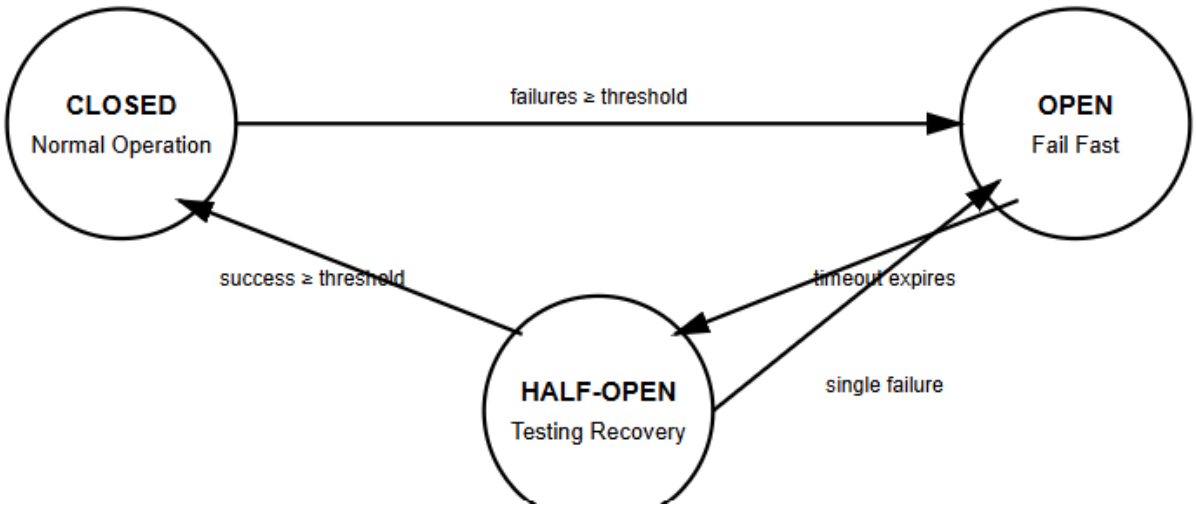
User Request → API Gateway → Circuit Breaker → OPEN (fails in <1ms)
Result: Instant response, no wasted resources, fallback used

Key Features

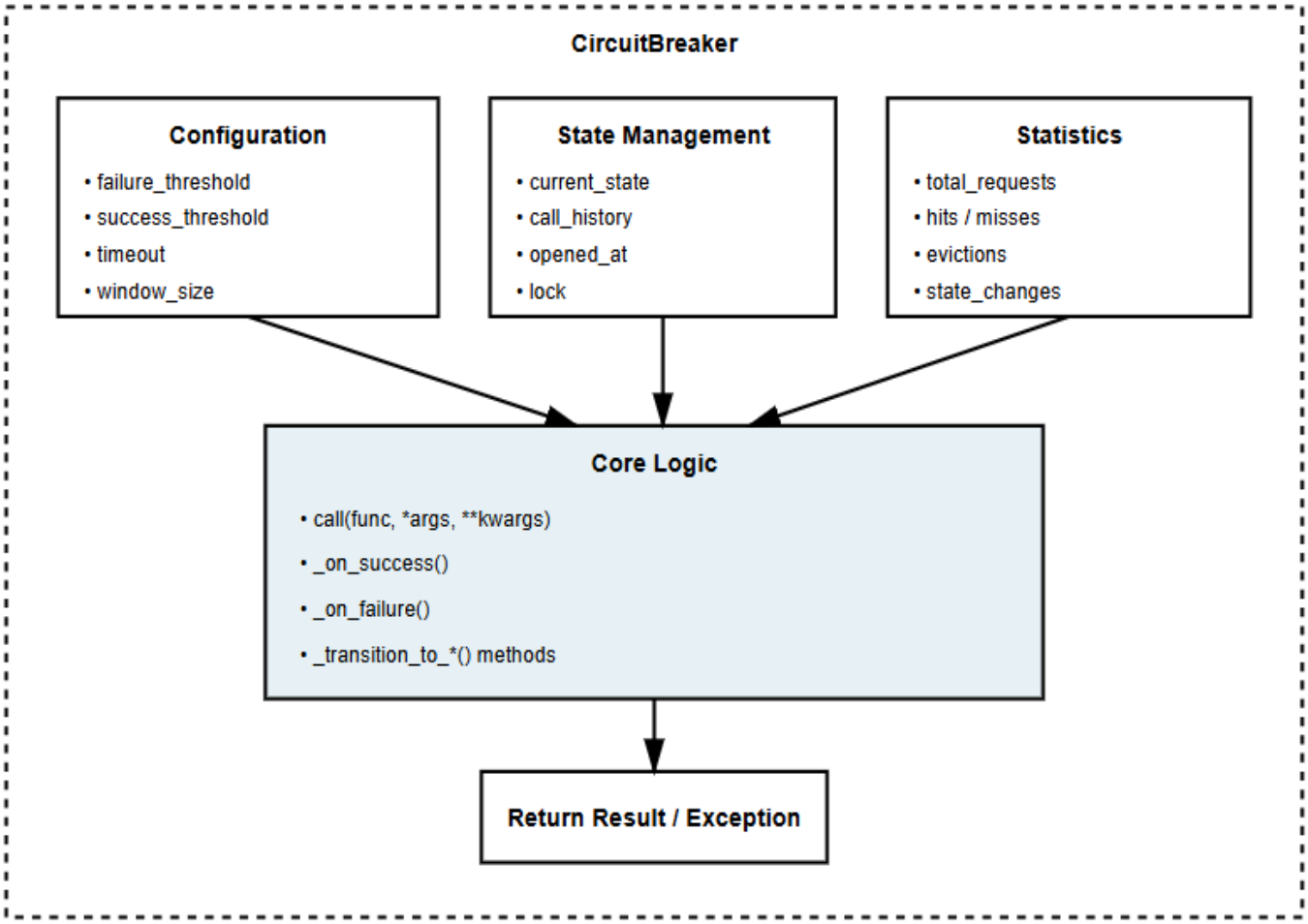
Feature	Description	Benefit
Fail-Fast	Reject requests when circuit is open	Prevent resource exhaustion
Auto-Recovery	Automatically test service health	No manual intervention needed
Thread-Safe	Safe for concurrent access	Works in multi-threaded apps
Sliding Window	Only recent failures matter	Accurate current health status
Monitoring	Real-time dashboard and metrics	Full observability
Configurable	Per-service settings	Tune for different SLAs

Architecture

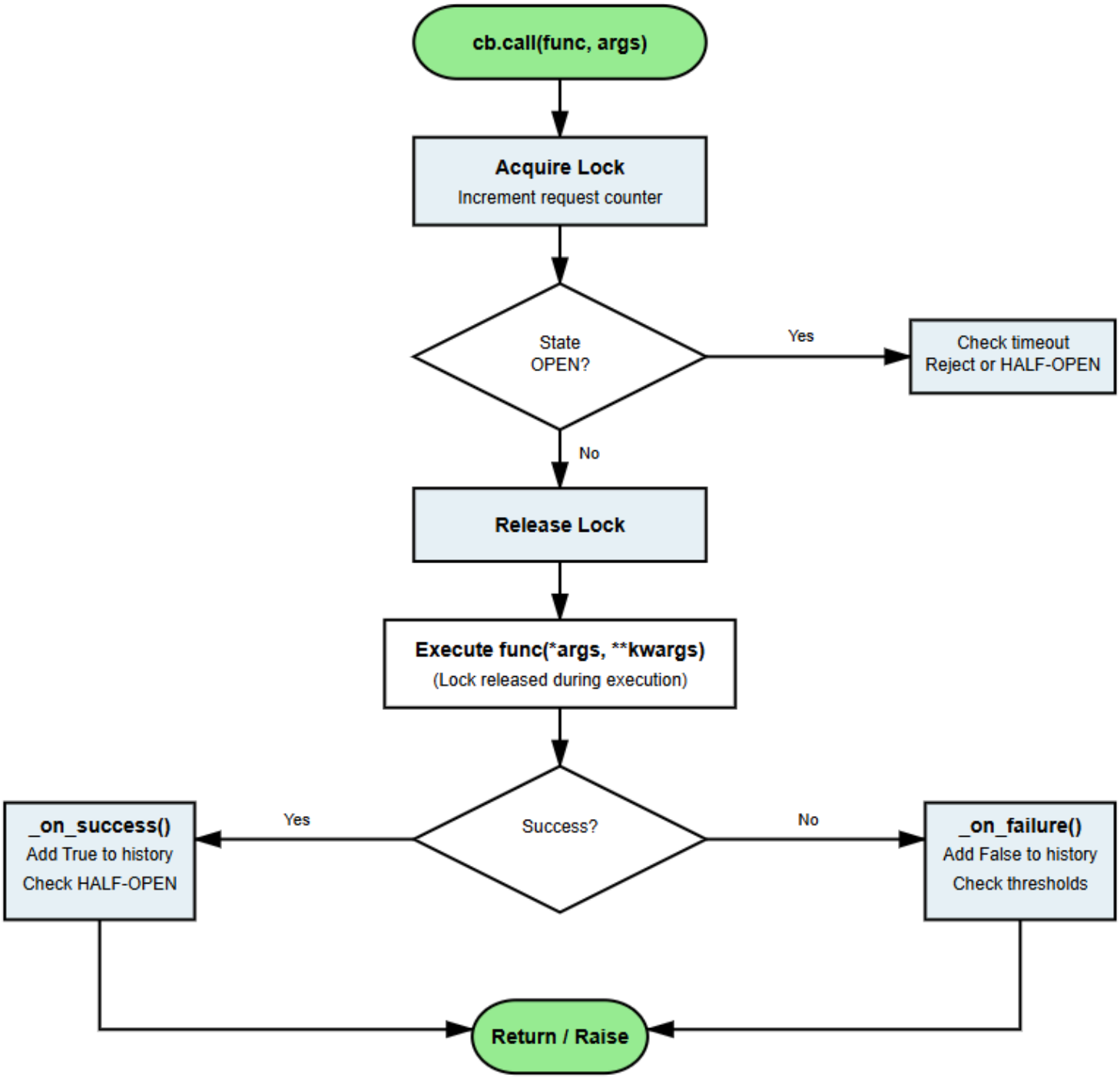
1. State Machine Diagram



2. Component Architecture



3. Execution Data Flow



Installation & Setup

Prerequisites

```
# Required
Python 3.7+

# Optional (for HTTP client only)
pip install requests
```

Project Structure

```
circuit-breaker/
├── circuit_breaker.py            # Core implementation
├── circuit_breaker_tests.py       # Unit tests
├── circuit_breaker_monitoring.py   # Monitoring & metrics
├── circuit_breaker_http.py        # HTTP client integration
├── README.md                      # Project overview
├── DOCUMENTATION.md               # This file
├── benchmarks.py                  # Performance tests
└── examples/                      # Usage examples
    ├── basic_usage.py
    ├── http_integration.py
    └── microservices_demo.py
```

Quick Installation

```
# Clone repository
git clone https://github.com/yourusername/circuit-breaker.git
cd circuit-breaker

# Verify installation
python circuit_breaker.py

# Run tests
python circuit_breaker_tests.py

# Expected output:
# Ran 25 tests in 2.345s
# OK
```

Quick Start Guide

Example 1: Protect a Flaky Function

```
from circuit_breaker import CircuitBreaker, CircuitBreakerConfig
import random

# Create circuit breaker
config = CircuitBreakerConfig(
    failure_threshold=3,  # Open after 3 failures
    success_threshold=2,  # Need 2 successes to close
    timeout=10.0,         # Wait 10 seconds before retry
    window_size=5         # Track last 5 calls
)
cb = CircuitBreaker(config)

# Your flaky function
def call_external_api():
    if random.random() < 0.3:  # 30% failure rate
        raise Exception("Service unavailable")
    return {"status": "success", "data": "..."}

# Use circuit breaker
for i in range(10):
    try:
        result = cb.call(call_external_api)
        print(f"✅ Call {i+1}: {result['status']}")
    except Exception as e:
        print(f"❌ Call {i+1}: {e}")

    # Check stats
    stats = cb.get_stats()
    print(f"   State: {stats['state']}, "
          f"Failures: {stats['failed']}/{stats['total_requests']}")
```

Example 2: HTTP API with Fallback

```
from circuit_breaker_http import ResilientHTTPClient
from circuit_breaker import CircuitBreakerOpenException
import json

# Setup
client = ResilientHTTPClient()
client.register_service(
    service_name='users_api',
    base_url='https://api.example.com'
)
```

```
# Cache for fallback
cache = {}

def get_user(user_id):
    """Get user with circuit breaker protection and cache fallback"""
    try:
        # Try real API
        response = client.get('users_api', f'/users/{user_id}')
        user_data = response.json()

        # Update cache
        cache[user_id] = user_data
        return user_data

    except CircuitBreakerOpenException:
        # Circuit is open - use cached data
        if user_id in cache:
            print(f"⚠️  Using cached data for user {user_id}")
            return cache[user_id]
        else:
            raise Exception("Service unavailable and no cache")

# Usage
user = get_user(123)
print(f"User: {user['name']}")
```

Example 3: Multiple Services with Monitoring

```
from circuit_breaker_monitoring import MonitoredCircuitBreaker
from circuit_breaker import CircuitBreakerConfig

# Create separate breakers for different services
payment_cb = MonitoredCircuitBreaker(
    config=CircuitBreakerConfig(
        failure_threshold=2,  # Strict for payments
        timeout=60.0
    ),
    service_name="payment_service",
    monitoring_port=8080
)

analytics_cb = MonitoredCircuitBreaker(
    config=CircuitBreakerConfig(
        failure_threshold=10,  # Lenient for analytics
        timeout=30.0
    ),
    service_name="analytics_service",
    monitoring_port=8081
```

```
)

# Start monitoring
payment_cb.start_monitoring()
analytics_cb.start_monitoring()

print("Dashboards:")
print("  Payments:  http://localhost:8080")
print("  Analytics: http://localhost:8081")

# Use circuit breakers...
```

API Reference

CircuitBreakerConfig

Configuration object for circuit breaker behavior.

```
@dataclass
class CircuitBreakerConfig:
    failure_threshold: int = 5
    success_threshold: int = 2
    timeout: float = 60.0
    window_size: int = 10
```

Parameters

Parameter	Type	Default	Description
`failure_threshold`	int	5	Number of failures to trigger circuit opening
`success_threshold`	int	2	Number of successes needed to close circuit (from HALF_OPEN)
`timeout`	float	60.0	Seconds to wait before attempting recovery (OPEN → HALF_OPEN)
`window_size`	int	10	Number of recent calls to track in sliding window

Example

```
# Strict configuration (opens quickly)
strict_config = CircuitBreakerConfig(
    failure_threshold=2,
    success_threshold=3,
    timeout=120.0,
```

```
    window_size=5
)


# Lenient configuration (tolerates more failures)
lenient_config = CircuitBreakerConfig(
    failure_threshold=10,
    success_threshold=2,
    timeout=30.0,
    window_size=20
)
```

CircuitBreaker

Main circuit breaker class.

Constructor

CircuitBreaker(config: CircuitBreakerConfig = None)

Parameters:

- `config` (optional): Configuration object. Uses defaults if not provided.

Example:

```
cb = CircuitBreaker()  # Use defaults
# OR
cb = CircuitBreaker(my_config)  # Use custom config
```

Methods

`call(func, *args, **kwargs)`

Execute a function through the circuit breaker.

Parameters:

- `func`: Callable to execute
- `*args`: Positional arguments for the function
- `**kwargs`: Keyword arguments for the function

Returns:

- Whatever `func` returns

Raises:

- `CircuitBreakerOpenException`: If circuit is open and request is rejected
- Any exception raised by `func`

Example:

```
# Simple function
result = cb.call(my_function)

# With arguments
result = cb.call(my_function, arg1, arg2, key=value)

# Lambda
result = cb.call(lambda x: x * 2, 5)  # Returns 10
```

Thread Safety: Safe to call from multiple threads

`get_stats()`

Get current circuit breaker statistics.

Returns: Dictionary with keys:

- `state` (str): Current state ("CLOSED", "OPEN", "HALF_OPEN")
- `total_requests` (int): Total calls made
- `successful` (int): Successful calls
- `failed` (int): Failed calls
- `rejected` (int): Rejected calls (circuit was open)
- `failure_rate` (str): Percentage of failures (e.g., "15.5%")
- `state_changes` (int): Number of state transitions
- `time_until_retry` (float): Seconds until retry (0 if not open)

Example:

```
stats = cb.get_stats()
print(f"State: {stats['state']}")
print(f"Success rate: {100 - float(stats['failure_rate'].rstrip('%'))}%")

if stats['state'] == 'OPEN':
    print(f"Retry in {stats['time_until_retry']:.0f} seconds")
```

`reset()`

Manually reset circuit breaker to CLOSED state.

Use cases:

- Testing
- Manual recovery after maintenance
- Emergency override

Example:

```
# Force reset
cb.reset()
print(cb.get_stats()['state'])  # "CLOSED"
```

Warning: Use sparingly. Let automatic recovery work in production.

CircuitBreakerOpenException

Exception raised when the circuit is open.

```
try:
    result = cb.call(api_function)
except CircuitBreakerOpenException as e:
    # Circuit is open - use fallback
    print(f"Circuit open: {e}")
    result = get_cached_data()
```

CircuitBreakerHTTPClient

HTTP client wrapper with circuit breaker protection.

Methods

`register_service(service_name, base_url, config=None)`

Register a downstream service.

Parameters:

- `service_name` (str): Unique identifier
- `base_url` (str): Base URL (e.g., "https://api.example.com")
- `config` (optional): Custom CircuitBreakerConfig

Example:

```
client = CircuitBreakerHTTPClient()
client.register_service(
    'payment_api',
```

```
    'https://api.stripe.com',
    CircuitBreakerConfig(failure_threshold=3)
)
```

`get(service_name, path, timeout=5.0, **kwargs)`

Make GET request.

Parameters:

- `service_name` (str): Registered service name
- `path` (str): API path (e.g., "/users/123")
- `timeout` (float): Request timeout in seconds
- `**kwargs`: Additional arguments for `requests.get()`

Returns: `requests.Response` object

Example:

```
response = client.get('payment_api', '/charges', timeout=10.0)
data = response.json()
```

Similar methods: `post()`, `put()`, `delete()`

`get_service_stats(service_name)`

Get stats for specific service.

Returns: Dictionary with circuit breaker statistics

Example:

```
stats = client.get_service_stats('payment_api')
print(f"Payment API state: {stats['state']}")
```

MonitoredCircuitBreaker

Circuit breaker with built-in HTTP monitoring.

Constructor

```
MonitoredCircuitBreaker(
    config: CircuitBreakerConfig = None,
    service_name: str = "api",
    monitoring_port: int = 8080
```

)

Methods

`start_monitoring()`

Start HTTP server for monitoring.

Endpoints created:

- `http://localhost:{port}/` - HTML dashboard
- `http://localhost:{port}/metrics` - Prometheus metrics
- `http://localhost:{port}/stats` - JSON statistics
- `http://localhost:{port}/health` - Health check

Example:

```
cb = MonitoredCircuitBreaker(monitoring_port=8080)
cb.start_monitoring()
# Now visit http://localhost:8080 in browser
```

`stop_monitoring()`

Stop HTTP server.

Configuration Guide

Choosing the Right Thresholds

Service Type	failure_threshold	success_threshold	timeout	window_size
Critical (payments, auth)	2 -3	3 - 5	60 - 120s	5 - 10
Important (user data, orders)	5	2 - 3	30 - 60s	10
Nice-to-have (analytics, recommendations)	10 - 20	2	15 - 30s	20
Best-effort (tracking, logs)	20+	2	10 -20s	30

Configuration Examples

High Availability Service

```
# Opens quickly, recovers conservatively
ha_config = CircuitBreakerConfig(
    failure_threshold=2,
    success_threshold=5,
    timeout=120.0,
    window_size=5
)
```

Load Shedding for Non-Critical Service

```
# Tolerates many failures, recovers quickly
shed_config = CircuitBreakerConfig(
    failure_threshold=20,
    success_threshold=2,
    timeout=15.0,
    window_size=30
)
```

Testing Configuration

```
# Fast cycling for demos
test_config = CircuitBreakerConfig(
    failure_threshold=2,
    success_threshold=1,
    timeout=5.0,
    window_size=3
)
```

Tuning Guidelines

1. **Start Conservative**
    - Low `failure_threshold` (3-5)
    - High `success_threshold` (3-5)
    - Long `timeout` (60s)
2. **Monitor and Adjust**
    - Too many false positives? → Increase `failure_threshold`
    - Circuit opens but service is fine? → Increase `window_size`
    - Takes too long to recover? → Decrease `timeout`
    - Opens too often? → Increase both thresholds
3. **Match Service SLA**

- 99.99% uptime service → strict thresholds
- 95% uptime service → lenient thresholds

Monitoring & Observability

Dashboard Overview

Access via `http://localhost:{port}/` when using `MonitoredCircuitBreaker`.

Features:

- Auto-refreshes every 2 seconds
- Color-coded state (🟢 CLOSED, 🟡 HALF_OPEN, 🔴 OPEN)
- Real-time metrics
- Countdown timer when circuit is open

Prometheus Metrics

Access via `http://localhost:{port}/metrics`

Available Metrics:

```
# State (0=CLOSED, 1=HALF_OPEN, 2=OPEN)
circuit_breaker_state{service="api"} 0

# Total requests
circuit_breaker_requests_total{service="api"} 1523

# Successful requests
circuit_breaker_requests_successful{service="api"} 1450

# Failed requests
circuit_breaker_requests_failed{service="api"} 50

# Rejected requests (circuit open)
circuit_breaker_requests_rejected{service="api"} 23

# Failure rate percentage
circuit_breaker_failure_rate_percent{service="api"} 3.28

# State transitions count
circuit_breaker_state_changes_total{service="api"} 4
```

Integration with Monitoring Tools

Grafana Dashboard

```
{
  "dashboard": {
    "title": "Circuit Breaker Monitoring",
    "panels": [
      {
        "title": "Circuit State",
        "targets": [
          {
            "expr": "circuit_breaker_state"
          }
        ]
      },
      {
        "title": "Request Rate",
        "targets": [
          {
            "expr": "rate(circuit_breaker_requests_total[5m])"
          }
        ]
      },
      {
        "title": "Failure Rate",
        "targets": [
          {
            "expr": "circuit_breaker_failure_rate_percent"
          }
        ]
      }
    ]
  }
}
```

Prometheus Alerting Rules

```
groups:
  - name: circuit_breaker_alerts
    rules:
      - alert: CircuitBreakerOpen
        expr: circuit_breaker_state == 2
        for: 5m
        annotations:
          summary: "Circuit breaker is open for {{ $labels.service }}"

      - alert: HighFailureRate
        expr: circuit_breaker_failure_rate_percent > 10
        for: 2m
        annotations:
          summary: "High failure rate for {{ $labels.service }}"
```

Testing Guide

Running Tests

```
# Run all tests
python circuit_breaker_tests.py

# Verbose output
python circuit_breaker_tests.py -v

# Run specific test class
python -m unittest circuit_breaker_tests.TestCircuitBreakerBasics

# Run single test
python -m unittest circuit_breaker_tests.TestCircuitBreakerBasics.test_circuit_opens_after_threshold_failures
```

Test Coverage

```
Test Suite Coverage:
├── Basic Functionality (8 tests)
│   ├── Initial state
│   ├── Successful calls
│   ├── Failed calls
│   ├── Circuit opening
│   ├── Request rejection
│   └── State transitions
│
├── Sliding Window (2 tests)
│   ├── Old failures eviction
│   └── Window size enforcement
│
├── Thread Safety (2 tests)
│   ├── Concurrent successful calls
│   └── Concurrent state transitions
│
├── Statistics (4 tests)
│   ├── Request tracking
│   ├── Failure rate calculation
│   ├── Time until retry
│   └── State change counting
│
└── Edge Cases (9 tests)
    ├── Exact threshold
    ├── Manual reset
    ├── Zero timeout
    ├── Large window sizes
    └── Boundary conditions
```

Writing Custom Tests

```
import unittest
from circuit_breaker import CircuitBreaker, CircuitBreakerConfig

class TestMyUsage(unittest.TestCase):
    def setUp(self):
        """Run before each test"""
        self.cb = CircuitBreaker(
            CircuitBreakerConfig(failure_threshold=2)
        )

    def test_my_scenario(self):
        """Test specific scenario"""
        # Arrange
        def failing_function():
            raise Exception("Test failure")

        # Act
        with self.assertRaises(Exception):
            self.cb.call(failing_function)

        # Assert
        stats = self.cb.get_stats()
        self.assertEqual(stats['failed'], 1)

if __name__ == '__main__':
    unittest.main()
```

Performance Benchmarks

Methodology

Tests run on:

- Python 3.11
- Single circuit breaker instance
- No actual I/O (in-memory function calls)

Results

```
Benchmark: 10,000 successful calls
──────────────────────────────────────────
Total time:        0.654 seconds
Requests/sec:      15,290
Overhead/request:  0.065 ms
Peak memory:       4.2 MB
Benchmark: 10,000 calls with 50% failures
```

```
──────────────────────────────────────────────────────────
Total time:        0.721 seconds
Requests/sec:      13,870
Overhead/request:  0.072 ms

Benchmark: Concurrent (100 threads, 100 calls each)
──────────────────────────────────────────────────────────
Total time:        3.456 seconds
Total requests:    10,000
Requests/sec:      2,894
Lock contention:   Minimal
Data races:        0
```

Running Benchmarks

```
# benchmarks.py
import time
from circuit_breaker import CircuitBreaker

cb = CircuitBreaker()

def noop():
    return "success"

# Benchmark
iterations = 10_000
start = time.perf_counter()

for _ in range(iterations):
    cb.call(noop)

elapsed = time.perf_counter() - start

print(f"Requests/sec: {iterations / elapsed:,.0f}")
print(f"Overhead: {(elapsed / iterations) * 1000:.3f}ms")
```

Performance Characteristics

Operation	Time Complexity	Space Complexity
`call()`	O(1)	O(1)
State check	O(1)	O(1)
Add to history	O(1)	O(window_size)

Failure count	O(window_size)	O(1)
State transition	O(1)	O(1)

Key insight: All operations are constant time except failure counting which is O(window_size), but window_size is typically small (5-20).

Troubleshooting

Common Issues

Issue: Circuit Opens Too Frequently

Symptoms:

- Circuit opens even though service seems healthy
- Frequent OPEN → CLOSED → OPEN cycles

Causes:

1. `failure_threshold` too low
2. `window_size` too small
3. Legitimate intermittent failures

Solutions:

```
# Increase thresholds
config = CircuitBreakerConfig(
    failure_threshold=10,  # Was 3
    window_size=20         # Was 5
)

# Or add retry logic
result = client.get_with_retry(
    'api',
    '/endpoint',
    max_retries=3
)
```

Issue: Circuit Never Opens

Symptoms:

- Service clearly failing but circuit stays CLOSED
- Many failures in stats but state is CLOSED

Causes:

1. Failures not raising exceptions
2. Catching exceptions before circuit breaker sees them
3. Window size too large

Solutions:

```
# Ensure exceptions are raised
def my_api_call():
    response = requests.get(url)
    if response.status_code >= 500:
        raise Exception("Server error")  # Must raise!
    return response

# Don't catch exceptions before circuit breaker
result = cb.call(my_api_call)  # Let cb see the exception
```

Issue: Circuit Takes Too Long to Recover

Symptoms:

- Circuit opens correctly but stays open for long time
- Service recovered but circuit still rejecting

Solution:

```
# Reduce timeout
config = CircuitBreakerConfig(
    timeout=15.0  # Was 60.0
)

# Or manually reset in emergencies
if manual_intervention_performed:
    cb.reset()
```

Issue: Thread Safety Concerns

Symptoms:

- Incorrect stats under load
- Unexpected state transitions

- Race condition warnings

Solution:

- This implementation IS thread-safe
- If seeing issues, verify you're not:
  - Sharing state outside circuit breaker
  - Modifying config after creation
  - Calling private methods directly

Debugging Checklist

```
# 1. Check current state
stats = cb.get_stats()
print(f"State: {stats['state']}")
print(f"Stats: {stats}")

# 2. Verify configuration
print(f"Config: {cb.config}")

# 3. Check call history
print(f"Recent calls: {list(cb.call_history)}")

# 4. Monitor state changes
initial_changes = cb.stats.state_changes
# ... do some calls ...
new_changes = cb.stats.state_changes
print(f"State changed {new_changes - initial_changes} times")

# 5. Test with simple function
def test_func():
    return "ok"

try:
    result = cb.call(test_func)
    print(f"Basic call works: {result}")
except Exception as e:
    print(f"Basic call failed: {e}")
```

Best Practices

1. Use Separate Circuit Breakers per Service

```
# Don't share one circuit breaker
cb = CircuitBreaker()
cb.call(payment_api)
```

```
cb.call(email_api)  # Payment failures affect email!

# Do use one per service
payment_cb = CircuitBreaker()
email_cb = CircuitBreaker()
payment_cb.call(payment_api)
email_cb.call(email_api)
```

2. Always Provide Fallbacks

```
# Don't just fail
try:
    data = cb.call(api_call)
except CircuitBreakerOpenException:
    raise  # User sees error!

# Do provide fallback
try:
    data = cb.call(api_call)
except CircuitBreakerOpenException:
    data = cache.get('fallback_data')
    log_warning("Using cached data")
```

3. Tune Based on Service SLA

```
# Critical service (99.99% uptime expected)
critical_config = CircuitBreakerConfig(
    failure_threshold=2,
    success_threshold=5,
    timeout=120.0
)

# Non-critical service (95% uptime acceptable)
noncritical_config = CircuitBreakerConfig(
    failure_threshold=15,
    success_threshold=2,
    timeout=30.0
)
```

4. Monitor State Changes

```
# Set up alerting
previous_state = cb.state

while True:
    current_state = cb.state
    if current_state != previous_state:
        alert(f"Circuit state changed: {previous_state} → {current_state}")
        previous_state = current_state
```

```
    time.sleep(1)
```

5. Log State Transitions

```
import logging

class LoggingCircuitBreaker(CircuitBreaker):
    def _transition_to_open(self):
        logging.warning(
            f"Circuit OPENED for {self.service_name}. "
            f"Failures: {self.stats.failed_requests}"
        )
        super()._transition_to_open()

    def _transition_to_closed(self):
        logging.info(f"Circuit CLOSED for {self.service_name}. Service recovered.")
        super()._transition_to_closed()
```

6. Test Circuit Breaker Behavior

```
# Include in integration tests
def test_handles_service_outage():
    # Simulate outage
    with mock_service_down():
        # Verify circuit opens
        for _ in range(config.failure_threshold):
            with pytest.raises(Exception):
                cb.call(service_call)

        assert cb.state == CircuitState.OPEN

        # Verify rejection
        with pytest.raises(CircuitBreakerOpenException):
            cb.call(service_call)
```

7. Use Health Checks

```
# Periodic health check
def health_check():
    stats = cb.get_stats()

    return {
        "healthy": stats['state'] != 'OPEN',
        "circuit_state": stats['state'],
        "failure_rate": stats['failure_rate']
    }

# In your web framework
@app.get("/health")
```

```
def get_health():
    health = health_check()
    status_code = 200 if health['healthy'] else 503
    return JSONResponse(health, status_code=status_code)
```

8. Combine with Retries

```
# Circuit breaker + exponential backoff
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=10)
)
def resilient_call():
    return cb.call(api_function)

# Now you have both fast-fail AND retry logic
```

Advanced Usage

Custom State Change Callbacks

```
class CallbackCircuitBreaker(CircuitBreaker):
    def __init__(self, config, on_open=None, on_close=None):
        super().__init__(config)
        self.on_open_callback = on_open
        self.on_close_callback = on_close

    def _transition_to_open(self):
        super()._transition_to_open()
        if self.on_open_callback:
            self.on_open_callback(self.get_stats())

    def _transition_to_closed(self):
        super()._transition_to_closed()
        if self.on_close_callback:
            self.on_close_callback(self.get_stats())

# Usage
def alert_on_open(stats):
    send_alert(f"Circuit opened! Failure rate: {stats['failure_rate']}")

def log_on_close(stats):
    logger.info(f"Circuit recovered after {stats['state_changes']} transitions")

cb = CallbackCircuitBreaker(
```

```
    config,
    on_open=alert_on_open,
    on_close=log_on_close
)
```

Distributed Circuit Breaker (Redis Backend)

```
import redis
import pickle
from datetime import timedelta

class DistributedCircuitBreaker(CircuitBreaker):
    """Circuit breaker with shared state in Redis"""

    def __init__(self, config, service_name, redis_client):
        super().__init__(config)
        self.service_name = service_name
        self.redis = redis_client
        self.state_key = f"cb:{service_name}:state"
        self.stats_key = f"cb:{service_name}:stats"

    def _load_state(self):
        """Load state from Redis"""
        state_data = self.redis.get(self.state_key)
        if state_data:
            return pickle.loads(state_data)
        return None

    def _save_state(self):
        """Save state to Redis"""
        state_data = {
            'state': self.state,
            'opened_at': self.opened_at,
            'call_history': list(self.call_history)
        }
        self.redis.setex(
            self.state_key,
            timedelta(seconds=self.config.timeout * 2),
            pickle.dumps(state_data)
        )

    def call(self, func, *args, **kwargs):
        # Load shared state
        shared_state = self._load_state()
        if shared_state:
            self.state = shared_state['state']
            self.opened_at = shared_state['opened_at']

        # Execute normally
```

```
        result = super().call(func, *args, **kwargs)

        # Save state
        self._save_state()

        return result

# Usage
redis_client = redis.Redis(host='localhost', port=6379)
cb = DistributedCircuitBreaker(config, 'api_service', redis_client)
```

Adaptive Thresholds

```
class AdaptiveCircuitBreaker(CircuitBreaker):
    """Automatically adjusts thresholds based on patterns"""

    def __init__(self, config):
        super().__init__(config)
        self.baseline_failure_rate = 0.05  # 5%

    def _on_success(self):
        super()._on_success()

        # If consistently succeeding, become more sensitive
        if self.stats.successful_requests > 100:
            recent_failures = sum(1 for x in self.call_history if not x)
            failure_rate = recent_failures / len(self.call_history)

            if failure_rate < self.baseline_failure_rate:
                # Lower threshold (open faster)
                self.config.failure_threshold = max(
                    2,
                    self.config.failure_threshold - 1
                )

    def _transition_to_closed(self):
        super()._transition_to_closed()

        # After recovery, reset to more conservative threshold
        self.config.failure_threshold = min(
            10,
            self.config.failure_threshold + 2
        )
```

Metrics Export to Multiple Backends

```
class MultiBackendExporter:
    """Export metrics to multiple monitoring systems"""
```

```
def __init__(self, circuit_breaker):
    self.cb = circuit_breaker

def export_to_statsd(self, statsd_client):
    """Export to StatsD/Datadog"""
    stats = self.cb.get_stats()
    statsd_client.gauge('circuit_breaker.state',
                        {'CLOSED': 0, 'HALF_OPEN': 1, 'OPEN': 2}[stats['state']])
    statsd_client.increment('circuit_breaker.requests', stats['total_requests'])
    statsd_client.increment('circuit_breaker.failures', stats['failed'])

def export_to_cloudwatch(self, cloudwatch_client):
    """Export to AWS CloudWatch"""
    stats = self.cb.get_stats()
    cloudwatch_client.put_metric_data(
        Namespace='CircuitBreaker',
        MetricData=[
            {
                'MetricName': 'FailureRate',
                'Value': float(stats['failure_rate'].rstrip('%')),
                'Unit': 'Percent'
            },
            {
                'MetricName': 'StateChanges',
                'Value': stats['state_changes'],
                'Unit': 'Count'
            }
        ]
    )

def export_to_influxdb(self, influx_client):
    """Export to InfluxDB"""
    stats = self.cb.get_stats()
    influx_client.write_points([{
        'measurement': 'circuit_breaker',
        'fields': {
            'state': stats['state'],
            'total_requests': stats['total_requests'],
            'failure_rate': float(stats['failure_rate'].rstrip('%'))
        }
    }])
```

Appendix

Related Patterns

Pattern	Purpose	When to Use
Circuit Breaker	Prevent cascading failures	External service calls
Retry	Recover from transient failures	Network hiccups
Bulkhead	Isolate resources	Prevent resource exhaustion
Rate Limiter	Control request rate	Protect from overload
Timeout	Prevent indefinite waits	Any I/O operation
Fallback	Provide alternative	User-facing services

Combining Patterns

Request Flow with Multiple Patterns:

User Request

- Rate Limiter (control incoming rate)
- Circuit Breaker (check service health)
- Timeout (prevent hanging)
- Retry with Backoff (handle transient failures)
- Bulkhead (isolate thread pool)
- Fallback (if all else fails)

Performance Tuning Checklist

- [ ] Separate circuit breakers per service
- [ ] Tune thresholds based on monitoring
- [ ] Use appropriate window size (5-20)
- [ ] Set timeout based on service recovery time
- [ ] Monitor state change frequency
- [ ] Test under load (100+ threads)
- [ ] Measure overhead (<1ms typical)
- [ ] Enable monitoring dashboard
- [ ] Set up alerts for OPEN state
- [ ] Document fallback strategies

Glossary

Circuit Breaker: Design pattern that prevents cascading failures by stopping calls to failing services.

CLOSED State: Normal operation, all requests pass through.

OPEN State: Circuit is "broken", requests are rejected immediately without calling the service.

HALF-OPEN State: Testing state where limited requests are allowed to check if service recovered.

Sliding Window: A fixed-size buffer that only remembers recent events, automatically forgetting old ones.

Failure Threshold: Number of failures that trigger circuit opening.

Success Threshold: Number of successes needed to close an open circuit.

Timeout: Time to wait before attempting recovery after the circuit opens.

Fast-Fail: Immediately reject requests instead of waiting for timeout.

Cascading Failure: When failure in one service causes failures in dependent services.

Support & Contributing

Getting Help

1. **Check Documentation**: You're reading it!
2. **Run Examples**: See `examples/` directory
3. **Read Tests**: Best source of usage examples
4. **Check Issues**: GitHub issues for known problems

Reporting Bugs

Include:

1. Python version
2. Configuration used
3. Minimal reproduction code
4. Expected vs actual behavior
5. Stack trace if applicable

Contributing

1. Fork the repository
2. Create feature branch
3. Write tests for new functionality
4. Ensure all tests pass
5. Update documentation

6. Submit pull request

Changelog

v1.0.0 (Current)

- Initial release
- Core circuit breaker implementation
- Thread-safe operations
- HTTP client integration
- Monitoring dashboard
- Prometheus metrics
- Comprehensive test suite
- Full documentation

Acknowledgments

- Inspired by Netflix Hystrix
- Pattern described by Michael Nygard in "Release It!"
- State machine design influenced by Martin Fowler's article

Last Updated: February 2026
Version: 1.0.0
Author: Aditya Pandey